

Logique et raisonnement

CM3 : Modélisation en logique propositionnelle (suite)

Catalin Dima, UPEC

2021

Puzzles logiques

Énoncés définissant des relations entre différentes entités.

Artistes peintres

Anna, Hélène et Clara sont trois peintres qui ont trois styles différents (impressionisme, surréalisme et cubisme) et utilisant trois types d'encres (huile, aquarelle et gouache).

- ▶ Hélène est la seule à utiliser le surréalisme.
- ▶ Le style impressioniste ne convient pas à l'aquarelle.
- ▶ Clara n'utilise pas les gouaches.
- ▶ Le surréalisme est utilisé avec les gouaches.
- ▶ Anna n'utilise pas l'aquarelle.

Utiliser ToulST pour résoudre ce puzzle.

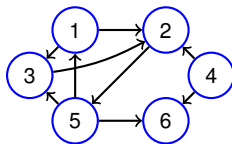
Problèmes de graphes

Étant donné un graphe orienté et deux noeuds, a et b , vérifier s'il existe un chemin entre les deux noeuds.

- ▶ Chemin = suite de noeuds.
- ▶ Taille du chemin $\leq$ nombre de noeuds dans le graphe.
- ▶ Propositions atomiques : $ch(i, j)$: le sommet i du graphe est le j -ième sommet du chemin.
- ▶ Contraintes données par les arêtes dans le graphe.
 - ▶ Le premier sommet du chemin est a et le dernier sommet est b : $ch(a, 0) \wedge ch(b, N)$.
 - ▶ Pour chaque $ch(i, j)$ doit être relié à $ch(k, j + 1)$ par une arête du graphe.
 - ▶ On accepte les boucles (i.e. le chemin peut boucler une ou plusieurs fois dans le même noeud) – pour régler l'inconnue de la longueur maximale du chemin.
- ▶ Formule dépendante du graphe donné.
- ▶ Pas de codage générique en ToulST, applicable à n'importe quel graphe !

Chemin dans graphe

On cherche s'il y a un chemin de 1 à 3 :



- ▶ Le chemin commence en 1 et s'arrête en 3 : $s(1, 0) \wedge s(3, \$N)$.
- ▶ Pour chaque position j dans le chemin, on trouve un voisin du noeud courant à la position suivante :

$$\bigwedge_{0 \leq j \leq 5} (s(1, \$j) \Rightarrow (s(2, \$j+1) \vee s(3, \$j+1) \vee s(1, \$j+1))) \wedge$$

$$(s(2, \$j) \Rightarrow (s(5, \$j+1) \vee s(2, \$j+1))) \wedge$$

$$(s(3, \$j) \Rightarrow (s(2, \$j+1) \vee s(3, \$j+1))) \wedge$$

$$(s(4, \$j) \Rightarrow (s(2, \$j+1) \vee s(6, \$j+1) \vee s(4, \$j+1))) \wedge$$

$$(s(5, \$j) \Rightarrow (s(3, \$j+1) \vee s(1, \$j+1) \vee s(6, \$j+1) \vee s(5, \$j+1)))$$

$$(s(6, \$j) \Rightarrow s(6, \$j+1))$$

- ▶ Notez qu'on peut boucler dans chaque noeud (sous-formule en rouge).
 - ▶ C'était pour ne pas deviner la longueur du chemin.
- ▶ Si on nous demande s'il y a un chemin de longueur 4 entre 1 et 3, alors il faut enlever les sous-formules qui représentent des boucles.
 - ▶ Du coup il n'y aurait plus de contrainte concernant le successeur de 6, car celui-ci n'a pas d'arête sortante.
- ▶ Notez que cette formule peut être utilisée pour d'autres paires de noeuds.

Autres applications

- ▶ Étant donné un automate fini et un mot, le mot est accepté par l'automate ?
 - ▶ Similaire au problème de chemin dans le graphe.
 - ▶ La longueur du mot est exactement la longueur du chemin.
 - ▶ Codage différent du chemin : les arêtes **sont étiquetées** !
 - ▶ Sur un chemin, les positions paires c'est les sommets et les positions impaires les étiquettes des arêtes.
- ▶ Étant donné un programme (même parallèle !), est-il possible de franchir une certaine configuration ?
 - ▶ Propositions atomiques représentant la valeur d'une variable à un instant i : $var(v, val, i)$.
 - ▶ Ne pas oublier la variable "compteur de programme" qui indique quelle est la ligne courante (celle qui devra s'exécuter prochainement).
 - ▶ Configuration = formule logique sur toutes ces variables.
 - ▶ Ça peut même être une formule qui parle de **suites** de valeurs de variables !
 - ▶ **Problème de vérification de programme**, approche par SAT-solving.

Problèmes de passage de rivière

Un fermier amenant un loup, une chèvre (vivants) et un chou, arrive sur le bord d'une rivière et veut passer avec les deux animaux et le chou de l'autre côté.

Un petit bateau est amarré pour permettre la traversée, mais ne possède que deux places, et à chaque traversée le fermier doit prendre place dans le bateau.

- ▶ Si le fermier laisse le loup et la chèvre seuls sur une berge, le loup mange la chèvre.
- ▶ Pareil, si la chèvre reste seule sur une berge avec le chou, elle mange le chou.

Quelle est la suite d'actions qui permet au fermier de passer tout de l'autre côté de la rivière ?

Modélisation des systèmes de transitions

- ▶ Jusqu'ici on a vu des modélisations de systèmes à contraintes logiques "statiques".
- ▶ Les problèmes de passage de rivière nécessitent une modélisation de **plusieurs situations** :
 - ▶ La situation initiale : qui est où ?
 - ▶ La situation après le premier passage, après le 2e passage etc.
- ▶ Le but n'est pas de coder une solution, mais de coder les contraintes et laisser ToulST trouver une solution :
 - ▶ Quelles contraintes pour la situation initiale ?
 - ▶ Quelles contraintes pour **chaque passage** ?
 - ▶ Quelles contraintes pour la situation **après chaque passage** ?
 - ▶ Quelles contraintes pour la situation au dernier passage ?

Quelles propositions atomiques pour le fermier, loup, la chèvre, le chou ?

- ▶ Première idée : le fermier/le loup/la chèvre/le chou est sur la rive gauche/droite : f, l, c, h .
- ▶ Mais ça ne représente que la situation à un certain moment !

Quelles propositions atomiques pour le fermier, loup, la chèvre, le chou ?

- ▶ Première idée : le fermier/le loup/la chèvre/le chou est sur la rive gauche/droite : f, l, c, h .
- ▶ Mais ça ne représente que la situation à un certain moment !
- ▶ Deuxième idée : à l'instant i , le fermier/le loup/la chèvre/le chou est sur la rive gauche/droite $f(\$i), l(\$i), c(\$i), h(\$i)$.
- ▶ Quel est l'intervalle pour $\$i$?
 - ▶ On peut considérer que dans maximum 7 passages tout le monde est de l'autre côté.
 - ▶ On triche ! Mais on y reviendra pour rediscuter ce paramètre.
- ▶ Quelle est la situation initiale ?
 - ▶ $\neg f(0) \wedge \neg l(0) \wedge \neg c(0) \wedge \neg h(0)$: tout le monde coté gauche.
- ▶ Et donc la situation finale :
 - ▶ $f(6) \wedge l(6) \wedge c(6) \wedge h(6)$: tout le monde coté droit.
- ▶ Pourquoi ne pas modéliser le bateau ?
 - ▶ Il suffit de signaler qui passe d'une rive à l'autre !

Contraintes "dynamiques" (1)

Contraintes après chaque passage :

- ▶ Le loup ne peut pas se trouver tout seul avec la chèvre **sur la rive droite** :
 $(l(\$i) \wedge c(\$i)) \Rightarrow f(\$i)$

Contraintes "dynamiques" (1)

Contraintes après chaque passage :

- ▶ Le loup ne peut pas se trouver tout seul avec la chèvre **sur la rive droite** :
 $(l(\$i) \wedge c(\$i)) \Rightarrow f(\$i)$
- ▶ Le loup ne peut pas se trouver tout seul avec la chèvre **sur la rive gauche** : $(\neg l(\$i) \wedge \neg c(\$i)) \Rightarrow \neg f(\$i)$

Contraintes "dynamiques" (1)

Contraintes après chaque passage :

- ▶ Le loup ne peut pas se trouver tout seul avec la chèvre **sur la rive droite** :
 $(l(\$i) \wedge c(\$i)) \Rightarrow f(\$i)$
- ▶ Le loup ne peut pas se trouver tout seul avec la chèvre **sur la rive gauche** : $(\neg l(\$i) \wedge \neg c(\$i)) \Rightarrow \neg f(\$i)$
- ▶ La chèvre ne peut pas se trouver toute seule avec le chou, soit sur la rive gauche, soit sur la rive droite
 $((c(\$i) \wedge h(\$i)) \Rightarrow f(\$i)) \wedge ((\neg c(\$i) \wedge \neg h(\$i)) \Rightarrow \neg f(\$i))$
- ▶ Toutes ces contraintes sont rassemblées en une grande conjonction.

Contraintes "dynamiques" (2)

Contraintes lors de chaque passage : au plus deux places dans le bateau !

Donc pour tout i :

- ▶ Soit le fermier traverse seul, les autres restent sur place :

$$(f(i) \oplus \neg f(i+1)) \wedge (l(i) \Leftrightarrow l(i+1)) \wedge (c(i) \Leftrightarrow c(i+1)) \wedge (h(i) \Leftrightarrow h(i+1))$$

- ▶ Soit le fermier traverse avec le loup de la rive gauche à la rive droite, les autres restent sur place :

$$\neg f(i) \wedge \neg l(i) \wedge f(i+1) \wedge l(i+1) \wedge (c(i) \Leftrightarrow c(i+1)) \wedge (h(i) \Leftrightarrow h(i+1))$$

- ▶ Soit le fermier traverse avec le loup de la rive droite à la rive gauche, les autres restent sur place :

$$f(i) \wedge l(i) \wedge \neg f(i+1) \wedge \neg l(i+1) \wedge (c(i) \Leftrightarrow c(i+1)) \wedge (h(i) \Leftrightarrow h(i+1))$$

- ▶ Soit le fermier traverse avec la chèvre de la rive gauche à la rive droite, les autres restent sur place :

$$\neg f(i) \wedge \neg c(i) \wedge f(i+1) \wedge c(i+1) \wedge (l(i) \Leftrightarrow l(i+1)) \wedge (h(i) \Leftrightarrow h(i+1))$$

- ▶ Etc....

Toutes ces contraintes sont rassemblées en une grande **disjonction**, qui est elle-même imbriquée dans une conjonction (**pour chaque i !**)

- ▶ Remarque : d'autres modélisations possibles.
- ▶ Implémentation en ToulST en TM.

Quel nombre maximum de passages ?

- ▶ On a triché, on savait que le fermier peut résoudre le problème en 7 passages !
- ▶ Mais si on ne le savait pas, comment on aurait pu trouver une valeur pour le nombre max de passages ?

Quel nombre maximum de passages ?

- ▶ On a triché, on savait que le fermier peut résoudre le problème en 7 passages !
- ▶ Mais si on ne le savait pas, comment on aurait pu trouver une valeur pour le nombre max de passages ?
- ▶ En essayant pour différentes valeurs !
 - ▶ Forcément c'est des entiers **impairs** !
 - ▶ Si échec pour un N impair, on essaye pour $N + 2$.

Quel nombre maximum de passages ?

- ▶ On a triché, on savait que le fermier peut résoudre le problème en 7 passages !
- ▶ Mais si on ne le savait pas, comment on aurait pu trouver une valeur pour le nombre max de passages ?
- ▶ En essayant pour différentes valeurs !
 - ▶ Forcément c'est des entiers **impairs** !
 - ▶ Si échec pour un N impair, on essaye pour $N + 2$.
- ▶ Remarque : si problème résolu en N passages, alors résolvable en $N + 2$, $N + 4$, $N + 6$, etc. passages !
 - ▶ Il suffit qu'à la fin le fermier fasse des allers-retours entre les deux rives !
 - ▶ Donc pour éviter trop d'échecs, dès le départ on peut mettre N assez grand.
 - ▶ Ensuite il faut identifier les passages redondants...

Quel nombre maximum de passages ?

- ▶ On a triché, on savait que le fermier peut résoudre le problème en 7 passages !
- ▶ Mais si on ne le savait pas, comment on aurait pu trouver une valeur pour le nombre max de passages ?
- ▶ En essayant pour différentes valeurs !
 - ▶ Forcément c'est des entiers **impairs** !
 - ▶ Si échec pour un N impair, on essaye pour $N + 2$.
- ▶ Remarque : si problème résolu en N passages, alors résolvable en $N + 2$, $N + 4$, $N + 6$, etc. passages !
 - ▶ Il suffit qu'à la fin le fermier fasse des allers-retours entre les deux rives !
 - ▶ Donc pour éviter trop d'échecs, dès le départ on peut mettre N assez grand.
 - ▶ Ensuite il faut identifier les passages redondants...

Limites de la modélisation en logique propositionnelle : pas de raisonnement avec des paramètres non-instanciés !

Problème d'exclusion mutuelle

- ▶ Processus parallèles qui ont besoin d'avoir accès à une ressource commune (même fichier, imprimante, etc.)
- ▶ Il faut éviter que plus d'un seul processus ait accès à la ressource en même temps (problème de cohérence).
 - ▶ Laisser un processus terminer son travail avec la ressource avant de permettre à un autre processus d'y accéder.
 - ▶ Mécanisme implémenté de différentes façons dans les langages de programmation.
 - ▶ Exemple : objets Java synchronisés.
 - ▶ Mais comment on implémente cela dans des langages de plus bas niveau ?
- ▶ "Synchroniser" des processus nécessite un mécanisme de partage d'informations (variables).
- ▶ Supposons un langage façon C ou des processus différents peuvent avoir accès à la même variable.
 - ▶ Exemple : segments de mémoire partagée en C.

Solution de l'exclusion mutuelle ?

Est-ce que l'idée suivante est une bonne solution à l'exclusion mutuelle ?

<u>flag1 := true ;</u>	<u>flag2 := true ;</u>
<u>wait $\neg$flag2 ;</u>	<u>wait $\neg$flag1 ;</u>
section critique 1	section critique 2
<u>flag1 := false ;</u>	<u>flag2 := false ;</u>

Petite explication

- ▶ **wait** est une instruction **bloquante** :
 - ▶ Si la condition est vraie, le processus avance à l'instruction d'après.
 - ▶ Sinon le processus n'avance pas.
- ▶ Elle peut être implémentée par un **while** ($\neg$ condition) ;
 - ▶ Si la condition est fausse on reste dans le while.
 - ▶ Ce n'est pas une boucle infinie car on a un autre processus qui peut changer les variables testées dans la condition !

Problème d'exclusion mutuelle

Que doit-on comprendre de cet exemple ?

<u>flag1 := true ;</u>	<u>flag2 := true ;</u>
<u>wait $\neg$flag2 ;</u>	<u>wait $\neg$flag1 ;</u>
<u>section critique 1</u>	<u>section critique 2</u>
<u>flag1 := false ;</u>	<u>flag2 := false ;</u>

- ▶ On a deux processus qui se partagent deux variables : flag1 et flag2.
- ▶ Les deux processus peuvent être exécutés sur une machine avec un seul processeur.
- ▶ Le seul processeur ne peut pas exécuter plusieurs instructions en même temps !
- ▶ Chaque processus possède un **compteur de programme (CP)** qui indique quelle instruction sera exécutée à un certain instant.
 - ▶ Le processeur choisit aléatoirement un des deux processus et exécute l'instruction pointée par le CP.
 - ▶ Une fois l'instruction terminée, le CP du processus est modifié pour pointer l'instruction suivante (qui peut ne pas être celle d'après pour le cas des if-then-else ou boucles).

Problème d'exclusion mutuelle

Questions :

1. Peut-on arriver à une situation où les deux CP pointent vers la section critique de chaque processus ?
2. Peut-on trouver les processus en **situation d'interblocage** ? (Le CP de chacun n'est jamais modifié même lorsqu'on exécute une instruction du processus respectif).

- | | |
|------------------------------|------------------------------|
| 1. <u>flag1 := true ;</u> | 1. <u>flag2 := true ;</u> |
| 2. <u>wait ¬flag2 ;</u> | 2. <u>wait ¬flag1 ;</u> |
| 3. <u>section critique 1</u> | 3. <u>section critique 2</u> |
| 4. <u>flag1 := false ;</u> | 4. <u>flag2 := false ;</u> |

- Pour répondre il faut essayer chaque exécution et voir ce qui se passe.
 - Un peu de réflexion nous amène à répondre par oui à la question 1 et non par la question 2 !
- **Utilisons ToulST pour cela !**

Exclusion mutuelle modélisée en ToulST (1)

- | | |
|---|---|
| 1. <u>flag1 := true ;</u> | 1. <u>flag2 := true ;</u> |
| 2. <u>wait $\neg$flag2 ;</u> | 2. <u>wait $\neg$flag1 ;</u> |
| 3. section critique 1 | 3. section critique 2 |
| 4. <u>flag1 := false ;</u> | 4. <u>flag2 := false ;</u> |

Variables propositionnelles :

- ▶ On considère 10 étapes de calcul sur le processus.
- ▶ On considère une instruction finale numérotée 5 pour chaque processus.
- ▶ $CP(\$i, \$j, \$k)$ pour $\$i = 1, 2$ (CP du premier ou 2e processus), $1 \leq \$j \leq 5$ (quelle est l'instruction courante) et $1 \leq \$k \leq 10$ (quelle étape de calcul).
- ▶ $tour(\$k)$ pour tous les $1 \leq \$k \leq 10$: lequel des 2 processus est choisi, à chaque instant $\$i$, pour avancer son calcul.
- ▶ $flag(\$i, \$k)$ pour tous les $\$i = 1, 2$ et $1 \leq \$k \leq 10$.

Exclusion mutuelle en ToulST (2)

Formules :

- ▶ Si c'est le tour du processus 1 et il est à la première instruction alors celle-ci s'exécute :

$$\bigwedge_{1 \leq k \leq 9} (tour(\$k) \wedge CP(1, 1, \$k)) \Rightarrow (CP(1, 2, \$k + 1) \wedge flag(1, \$k + 1))$$

- ▶ Pareil pour le processus 2 :

$$\bigwedge_{1 \leq k \leq 9} (\neg tour(\$k) \wedge CP(2, 1, \$k)) \Rightarrow (CP(2, 2, \$k + 1) \wedge flag(2, \$k + 1))$$

- ▶ Si c'est le tour du processus 1 et il est à la deuxième instruction alors celle-ci s'exécute :

$$\begin{aligned} & \left(\bigwedge_{1 \leq k \leq 9} \left((tour(\$k) \wedge CP(1, 2, \$k) \wedge flag(2, \$k)) \Rightarrow \right. \right. \\ & \quad \left. \left. (CP(1, 2, \$k + 1) \wedge (flag(1, \$k) \Leftrightarrow flag(1, \$k + 1))) \right) \right) \wedge \\ & \left(\bigwedge_{1 \leq k \leq 9} (tour(\$k) \wedge CP(1, 2, \$k) \wedge \neg flag(2, \$k)) \Rightarrow \right. \\ & \quad \left. (CP(1, 3, \$k + 1) \wedge (\neg flag(1, \$k) \Leftrightarrow flag(1, \$k + 1))) \right) \end{aligned}$$

- ▶ Pareil pour le processus 2.

Exclusion mutuelle en ToulST (3)

- ▶ Si c'est le tour du processus 1 qui est à la 3e instruction alors celle-ci s'exécute :

$$\bigwedge_{1 \leq k \leq 9} (tour(k) \wedge CP(1, 3, k)) \Rightarrow (CP(1, 4, k+1) \wedge (flag(1, k) \Leftrightarrow flag(1, k+1)))$$

- ▶ Si c'est le tour du processus 1 qui est à la 4e instruction alors celle-ci s'exécute :

$$\bigwedge_{1 \leq k \leq 9} (tour(k) \wedge CP(1, 4, k)) \Rightarrow (CP(1, 5, k+1) \wedge \neg flag(1, k+1))$$

- ▶ Pareil pour les 2 autres instructions et le processus 2.
- ▶ Si c'est pas le tour du processus 1, son CP et son flag restent inchangés :

$$\bigwedge_{1 \leq k \leq 9} \bigwedge_{1 \leq j \leq 5} (\neg tour(k) \Rightarrow (CP(1, j, k) \Leftrightarrow CP(1, j, k+1)) \wedge (flag(1, k) \Leftrightarrow flag(1, k+1)))$$

- ▶ Pareil pour le processus 2.

Exclusion mutuelle en ToulST (4)

- ▶ Le compteur de programme de chaque processus ne peut avoir qu'une seule valeur :

$$\bigwedge_{i=1,2} \bigwedge_{1 \leq k \leq 9} \text{exact}(1, CP(i, [1..5], k))$$

- ▶ Au début les deux compteurs pointent l'instruction initiale :

$$CP(1, 1, 1) \wedge CP(2, 1, 1)$$

- ▶ Peut-on avoir un comportement sans exclusion mutuelle (les deux processus sont en train d'exécuter leur "section critique" en même temps) ?

$$\bigvee_{1 \leq k \leq 9} (CP(1, 3, k) \wedge CP(2, 3, k))$$

- ▶ Peut-on avoir des comportements où à partir d'une étape aucun processus ne peut avancer (interblocage) ?

$$\bigvee_{1 \leq i \leq 8} \bigwedge_{i \leq k \leq 9} CP(1, i, k) \wedge CP(1, i, k+1) \wedge CP(2, i, k) \wedge CP(2, i, k+1)$$

- ▶ Tester les deux questions **séparément** !